



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**



TRANSFORMACIÓN
digital

CURSOS GRATUITOS

OTI UNI

Git y GitHub desde cero

Programa Completo – Git y GitHub (9na Edición)

Instructor:  [Anthony Gonzalo Quispe Cordova](#)

Oficina de Tecnologías de la Información (OTI)

22 de mayo de 2026



Git + GitHub

Control de versiones y colaboración

Flujo de Cursos Orientado a DevOps



Horarios de los Cursos del Flujo DevOps



Fundamentos de Linux



Sábados



9:00 a.m. – 12:00 p.m.



23/05 – 27/06 | 18 h



Git y GitHub



Lun, Mar, Jue, Vie



4:00 – 7:00 p.m.



19/05 – 28/05 | 18 h



Fundamentos de Docker



Lun, Mar, Vie



4:00 – 7:00 p.m.



29/05 – 09/06 | 18 h



Ansible: Automatización



Sábados



2:00 – 5:00 p.m.



23/05 – 27/06 | 18 h

Índice General

1 Introduccion y fundamentos

2 Entorno Git: instalacion y configuracion

3 Flujo local con Git

4 Ramas, merge y conflictos

5 Historial y deshacer cambios

6 Rebase y limpieza de historial

Introduccion y fundamentos

Bienvenida y contexto

- El objetivo de esta sesión es entender qué problema resuelve Git antes de memorizar comandos.
- Trabajaremos con ejemplos locales para construir una base sólida antes de pasar a colaboración remota.
- Al final de la sesión tendrás tu primer repositorio funcionando desde cero.

Requisitos previos

Conocimientos

- Manejo básico de archivos y carpetas.
- Nociones simples de terminal.
- Ganas de practicar, equivocarse y corregir.

Herramientas

- Git instalado o listo para instalar.
- Editor de código.
- Terminal: Git Bash, PowerShell, CMD, macOS Terminal o Linux Shell.

¿Qué es un control de versiones?

Un control de versiones guarda la historia de un proyecto para saber **qué cambió, cuándo y por qué.**

🕒 Historial

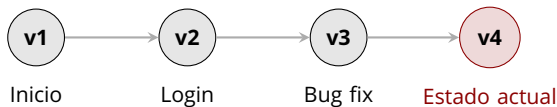
Registra cambios importantes en una línea de tiempo.

🔄 Recuperación

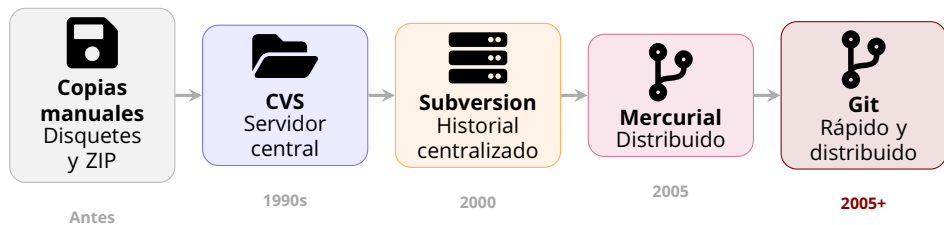
Permite volver a un punto anterior si algo falla.

🔍 Trazabilidad

Ayuda a responder quién cambió qué y cuándo.



Evolución de los sistemas de control de versiones



Idea central

La evolución fue desde copiar archivos manualmente hacia sistemas que guardan historial, facilitan colaboración y permiten trabajar sin depender siempre de un servidor central.

¿Por qué es importante?



Seguridad

Recuperas versiones anteriores sin depender de copias manuales.



Orden

Cada cambio queda documentado en una línea de tiempo clara.



Colaboración

Varias personas trabajan sobre el mismo proyecto con reglas claras.

¿Qué es Git?

- Git es un sistema de control de versiones **distribuido**.
- Cada repositorio local contiene el historial del proyecto.
- Fue creado por **Linus Torvalds** para gestionar proyectos grandes y cambiantes.
- No necesitas internet para guardar cambios localmente con `commit`.



Linus Torvalds

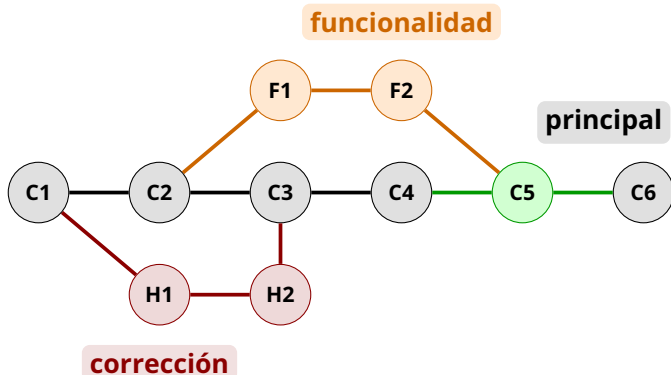
Vista previa: ¿qué es una rama?

- Una rama es una línea de trabajo dentro del historial del proyecto.
- Permite separar cambios: una idea, una corrección o una nueva funcionalidad.
- En esta sesión solo veremos la idea; el trabajo práctico con ramas vendrá después.

Ejemplo mental

La rama principal conserva la versión estable; una rama secundaria permite experimentar sin afectar esa línea estable.

Visualizando ramas en Git



Las ramas ayudan a trabajar cambios separados y luego integrarlos al historial principal.

master y main: nombres de la rama principal

- Ambas suelen representar la rama principal del proyecto.
- Algunos repositorios antiguos usan `master`; muchos proyectos nuevos usan `main`.
- En esta sesión solo necesitamos reconocer el nombre que Git muestre al inicializar el repositorio.
- El manejo práctico de ramas se trabajará más adelante.

¿Git y GitHub son lo mismo?



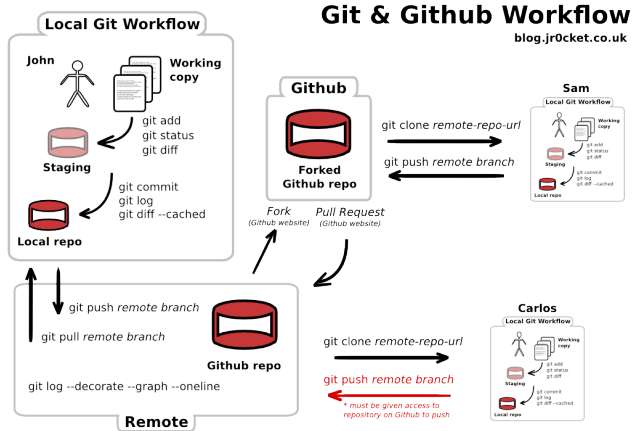
Git

- Herramienta instalada en tu equipo.
- Controla versiones localmente.
- Funciona sin internet.

GitHub

- Plataforma web para alojar repositorios.
- Facilita colaboración, revisión y publicación.
- Usa Git como base.

Relación entre Git y GitHub



Linux y la terminal: base del trabajo moderno

- Linux está presente en servidores, nube, contenedores y automatización.
- La terminal permite trabajar de forma rápida, repetible y precisa.
- Git se usa mucho desde línea de comandos porque muestra claramente el estado del proyecto.
- Dominar lo básico facilita avanzar hacia Docker, Ansible y CI/CD.

Idea clave

No necesitamos ser expertos en Linux para usar Git, pero sí movernos con confianza en la terminal.



Linux + Terminal

Base práctica para Git y DevOps

Manos a la obra

Practica comandos básicos de Linux directamente en el navegador:

<https://kodekloud.com/studio/labs/linux/>

Familiarizarse con la terminal ayuda a entender mejor los comandos de Git.



Guía rápida oficial de comandos Ubuntu

Usa este cheatsheet como apoyo para practicar navegación, archivos, paquetes, usuarios y red en la terminal.



Descargar / abrir PDF

https://assets.ubuntu.com/v1/d00791ae-ubuntu_cli_cheat_sheet_2025.pdf

Entorno Git: instalacion y configuracion

¿Cómo saber si tengo Git instalado?

Comando de verificación

```
git --version
```

- Si aparece una versión, Git está instalado.
- Si el sistema no reconoce el comando, necesitas instalarlo o revisar el PATH.

Instalando Git en tu sistema



Windows

Descargar el instalador
standalone desde:

git-scm.com



macOS

Usar Homebrew:

```
brew install git
```

O el instalador de
Xcode.

[descarga oficial](#)



Linux

Basado en Debian:

```
sudo apt install git
```

Basado en Fedora:

```
sudo dnf install git
```

Verificación final

Abre una terminal y escribe: `git --version`

Configurar nombre y correo en Git

Identidad global

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@email.com"
```

- Git usa estos datos para firmar cada commit.
- Usa el correo que quieras asociar a tus proyectos.
- La opción `-global` aplica la configuración a todos tus repositorios del usuario actual.

Configurar el editor por defecto

Ejemplo con Visual Studio Code

```
git config --global core.editor "code --wait"
```

- El editor se abre cuando Git necesita que escribas mensajes o resuelvas operaciones.
- Puedes usar otro editor si es el que tienes instalado y dominas.

Comprobar configuración de Git

Ver toda la configuración

```
git config --list
```

Consultar un valor específico

```
git config user.name  
git config user.email
```

Flujo local con Git

¿Cómo inicializar un nuevo proyecto Git?

Crear un repositorio local

```
mkdir mi-proyecto  
cd mi-proyecto  
git init
```

- `git init` crea la carpeta oculta `.git`.
- Desde ese momento Git puede registrar el historial del proyecto.

¿Qué es el directorio de trabajo?

- Es la carpeta visible donde editas, creas y borras archivos.
- Git compara ese estado contra el último commit conocido.
- Cuando un archivo cambia, Git puede verlo como modificado o sin seguimiento.

Los tres estados en Git



Idea clave

Primero modificas, luego preparas lo que quieres guardar y finalmente confirmas el cambio.

Los 3 Estados de Git

El ciclo básico de los archivos en Git



Working Directory
(modified)

Aquí editas tus archivos.
Los cambios aún no están
preparados.



`git add`



Staging Area
(staged)

Los cambios están preparados
y listos para confirmar.



`git commit`



Local Repository
(committed)

Los cambios están guardados
de forma permanente en tu
repositorio local.

Revisar el estado con `git status`

Comando esencial

```
git status
```

- Muestra archivos modificados, preparados y sin seguimiento.
- También indica si el repositorio está limpio o si hay cambios pendientes.

Añadir archivos al área de staging

Preparar cambios

```
git add archivo.txt  
git add .
```

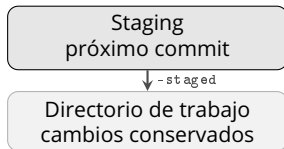
- `git add archivo.txt` prepara un archivo específico.
- `git add .` prepara todos los cambios del directorio actual.
- Staging permite elegir exactamente qué entrará en el próximo commit.

Sacar archivos del área de staging

Quitar del próximo commit sin borrar el trabajo

```
git restore --staged archivo.txt
```

- Saca el archivo del área de staging.
- Tus cambios **siguen existiendo** en el directorio de trabajo.
- Úsalo si hiciste `git add` por error o quieres separar commits.



¿Qué es un commit?

- 📷 Instantánea confirmada del estado del proyecto.
- 🔑 Tiene un identificador único (SHA).
- 👤 Registra autor, fecha y mensaje.
- 📍 Es un punto al que puedes volver.

Regla práctica

Si no puedes resumir el cambio en una frase clara, el commit es demasiado grande.



Un commit es como una foto: congela un estado claro del proyecto.

Haz commits atómicos

- ⚙️ Un commit = un solo cambio lógico.
- 👁️ Facilita revisar y entender el historial.
- ↶ Permite revertir cambios con precisión.
- 🚫 Evita mezclar tareas distintas en un mismo commit.

✓ Atómico

Agrega validación del formulario de registro

✗ No atómico

Agrega login, corrige CSS y actualiza readme

¿Cómo hacer un commit?

Flujo completo

```
git add archivo.txt  
git status  
git commit -m "Describe el cambio"
```

- ✓ Revisa el estado antes de confirmar.
- ✎ El mensaje debe ser claro y en modo imperativo.

👍 Buenos mensajes

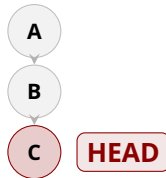
Agrega botón de login
Corrige validación de email
Actualiza dependencias

👎 Evita

cambios, fix, asdf, wip

¿Qué es HEAD?

- 📌 HEAD es un puntero a tu posición actual.
- 🔗 Normalmente apunta al último commit de la rama activa.
- ⚙️ Muchos comandos usan HEAD para comparar o restaurar.



¿Cómo puedo deshacer mis cambios?

1. Solo modificado

```
git restore archivo.txt
```

Descarta cambios locales del archivo.

2. En staging

```
git restore -staged  
archivo.txt
```

Lo quita del próximo commit, pero conserva el trabajo.

3. Ya hice commit

Usa `git revert` o crea un commit correctivo. Evita borrar historial compartido.

Regla de seguridad

Antes de deshacer, ejecuta `git status`. Si el cambio no está en un commit, `git restore archivo.txt` puede perderlo.

Ignorar archivos con .gitignore

- .gitignore indica qué archivos no debe rastrear Git.
- Se usa para dependencias, archivos temporales, binarios y secretos locales.

Ejemplo

```
node_modules/  
*.log  
.env
```

Ignorar siempre los mismos archivos

- Algunos archivos dependen de tu sistema o editor y no deberían repetirse en cada proyecto.
- Para eso puedes usar un `.gitignore` global.

Casos comunes

Archivos del sistema operativo, configuraciones personales del editor y temporales locales.

Comandos básicos en el ciclo de vida

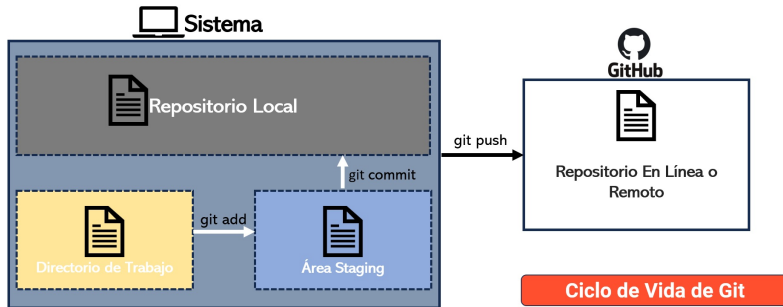
Máster en Git y GitHub Moderno



git



GitHub



Laboratorio: primer flujo local

- 1 Crea una carpeta e inicializa Git con **git init**.
- 2 Crea **README.md** y revisa el estado con **git status**.
- 3 Prepara el archivo con **git add README.md**.
- 4 Crea un commit con **git commit -m 'Agrega README inicial'**.
- 5 Modifica el archivo, deshaz el cambio y vuelve a revisar el estado.
- 6 Crea **.gitignore** e ignora archivos ***.tmp**.

Meta

Recorrer el ciclo básico: modificar → preparar → confirmar → revisar.

Resumen de la sesión

-  Git: control de versiones distribuido.
-  Flujo: modificar  preparar  confirmar.
-  La config define tu identidad global.
-  `git status`: tu brújula principal.
-  Commits atómicos = historial legible.
-  `.gitignore`: mantén limpio el repo.

Próxima sesión

Historial y control avanzado: `git diff`, `git log` detallado y cómo recuperar errores.

Ramas, merge y conflictos

Que vamos a resolver hoy?

Situacion real

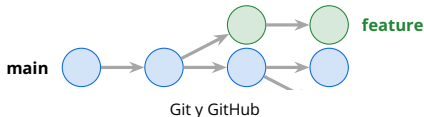
En la sesion anterior trabajamos de forma lineal: crear archivos, preparar cambios y hacer commits.

Nuevo problema

En proyectos reales varias ideas avanzan al mismo tiempo: una funcionalidad, un bug, una prueba y documentacion.

Respuesta de Git

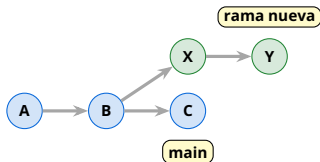
Las ramas permiten trabajar en paralelo y luego decidir que cambios se integran al proyecto principal.



De una linea recta al trabajo paralelo

Una rama es una linea de evolucion que nace desde un commit y permite trabajar sin tocar directamente `main`.

- Antes trabajamos de forma lineal: cambio, staging y commit.
- En proyectos reales varias ideas avanzan al mismo tiempo.
- Las ramas permiten aislar cambios y fusionarlos cuando esten listos.



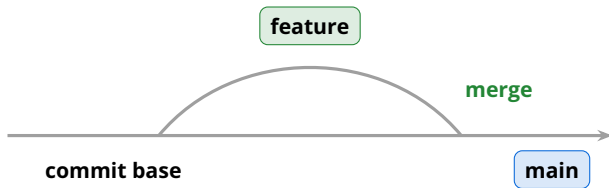
La analogia correcta

El arbol confunde un poco

En un arbol real, una rama nace del tronco, pero normalmente no vuelve a unirse al tronco.

Mejor: una autopista

Una rama de Git se parece mas a una salida paralela: te separas por un tramo y mas adelante puedes volver a la via principal.



Saber donde estamos

Ver todas las ramas locales

```
git branch
```

Git marca con * la rama activa.

Mostrar solo la rama actual

```
git branch --show-current
```

Util antes de hacer commits o fusiones.

Regla practica

Antes de moverte o fusionar, revisa `git status`. Git trabaja con estados y necesita saber que esta confirmado y que sigue pendiente.

Crear y moverse entre ramas

Crear primero, entrar despues

```
git branch fix  
git switch fix
```

Crear y entrar en un paso

```
git switch -c mi-primer-rama
```

Detalle importante

Para crear una rama debe existir al menos un commit. Si el repositorio esta recién inicializado y no tiene historial, Git no tiene un commit base desde donde sostener la rama.

Escenario: web de Cafe Aroma

Proyecto

Tu y otra persona desarrollan la web de **Cafe Aroma**.

`main`

Representa la version estable: lo que deberia poder mostrarse sin sorpresas.

Ramas temporales

Cada cambio importante vive en su propia rama:

- menu de productos,
- estilos visuales,
- encabezado principal.

1. Crear el proyecto base

```
mkdir cafe-aroma  
cd cafe-aroma  
git init -b main  
  
touch index.html estilos.css menu.txt
```

Que acaba de pasar

Git creo un repositorio vacio, una rama principal llamada main y una carpeta oculta .git.

Que guarda .git

Historial, commits, ramas y referencias internas del repositorio.

2. Primer estado confirmado

```
# index.html
<h1>Cafe Aroma</h1>

# menu.txt
- Cafe americano
- Te

git status
git add .
git commit -m "Proyecto inicial de cafeteria"
```

Git piensa en estados

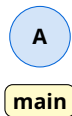
Los archivos pasan por:

Working Directory → Staging Area → Repository

Commit

No es una carpeta: es una fotografia exacta del proyecto en un punto del tiempo.

Despues del primer commit



Lectura

A representa el primer commit: la version inicial de Cafe Aroma.

3. Rama para el menu

```
git switch -c feature-menu
```

```
# editar menu.txt
```

- Cafe americano
- Te
- Capuccino
- Cheesecake

```
git add menu.txt
```

```
git commit -m "Agregar nuevos productos"
```

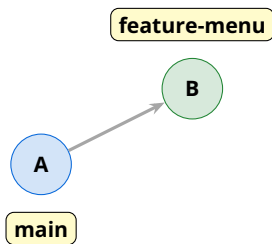
Que hizo Git

No copio todo el proyecto. Creo un nuevo puntero que nacio en el commit actual.

Idea clave

main no cambio. La funcionalidad del menu evoluciono aislada.

Menu aislado en su rama



Al volver a `main`

El cheesecake desaparece porque estas viendo la fotografia del commit A. Git no lo borro: esta guardado en otra rama.

4. Otra rama: estilos

```
git switch main
git switch -c feature-estilos

# estilos.css
body {
  background-color: beige;
}

git add estilos.css
git commit -m "Agregar estilos iniciales"
```

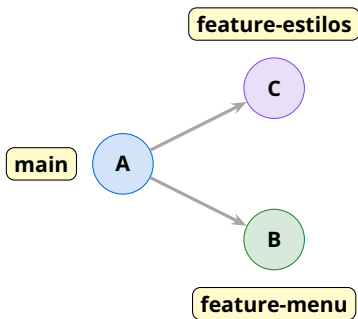
Que representa

Ahora existen dos lineas de evolucion distintas: una para menu y otra para estilos.

Ventaja

Varias personas pueden avanzar sin pisarse, siempre que separen bien las tareas.

Dos líneas de trabajo



Lectura

Ambas ramas nacieron desde el mismo commit, pero guardan cambios distintos.

5. Fusionar cambios sin conflicto

```
git switch main  
git merge feature-estilos  
git merge feature-menu
```

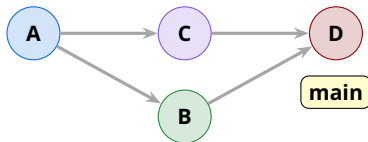
Por que funciona

Una rama modifiko `estilos.css`; la otra modifiko `menu.txt`. Git puede combinar cambios en zonas diferentes.

Idea

Fusionar significa traer a la rama actual los cambios que vivian en otra rama.

Historial despues de fusionar



Lectura

`main` vuelve a concentrar el trabajo terminado: estilos y menu ya forman parte de la version estable.

Cuando Git no puede decidir

Un conflicto aparece cuando dos ramas modifican la misma zona de un archivo de formas incompatibles.

Rama `feature-header`

Cambia el titulo a: Bienvenidos a Cafe Aroma

Rama `main`

Cambia la misma linea a: Cafe Aroma - El mejor cafe de Lima

Idea clave

Git no toma decisiones humanas. Cuando no sabe que version conservar, se detiene y pide intervencion.

6. Provocar el conflicto

```
git switch -c feature-header
```

```
# index.html
```

```
<h1>Bienvenidos a Cafe Aroma</h1>
```

```
git add index.html
```

```
git commit -m "Cambiar encabezado"
```

```
git switch main
```

```
# index.html
```

```
<h1>Cafe Aroma - El mejor cafe de Lima</h1>
```

```
git add index.html
```

```
git commit -m "Agregar slogan principal"
```

```
git merge feature-header
```

Marcas de conflicto

Git deja el archivo con una comparacion explicita:

```
<<<<<< HEAD
<h1>Cafe Aroma - El mejor cafe de Lima</h1>
=====
<h1>Bienvenidos a Cafe Aroma</h1>
>>>>>> feature-header
```

- HEAD: lo que existe en la rama actual, en este caso `main`.
- `feature-header`: lo que viene desde la rama que estamos fusionando.
- La linea `=====` separa ambas propuestas.

7. Resolver conscientemente

Decision humana

Combinamos ambas ideas:

```
<h1>Bienvenidos a Cafe Aroma - El mejor cafe de Lima</h1>
```

Finalizar la resolucion

```
git add index.html  
git commit -m "Resolver conflicto del encabezado"
```

Que significa `git add` aqui

No solo prepara un archivo: le dice a Git que el conflicto ya fue resuelto.

8. Eliminar ramas ya fusionadas

Despues del merge

```
git branch -d feature-menu  
git branch -d feature-estilos  
git branch -d feature-header
```

Por que limpiar

Las ramas eran lineas temporales de trabajo. Si ya cumplieron su proposito, mantenerlas genera ruido visual.

Proteccion de Git

-d elimina solo si la rama ya fue fusionada. Si no lo fue, Git te avisa para evitar perder commits.

Cuando una rama no fue fusionada

Mensaje tipico

Si Git dice que la rama no esta completamente fusionada, te esta protegiendo de perder commits.

Eliminar con proteccion

```
git branch -d nombre-rama
```

Forzar eliminacion

```
git branch -D nombre-rama
```

La D mayuscula significa: eliminar aunque se pierdan cambios.

Meta final: al listar ramas, solo deberia quedar `main`.

Laboratorio: Cafe Aroma

```
mkdir cafe-aroma
cd cafe-aroma
git init -b main

echo "<h1>Cafe Aroma</h1>" > index.html
echo "- Cafe americano" > menu.txt
touch estilos.css
git add .
git commit -m "Proyecto inicial de cafeteria"

git switch -c feature-menu
echo "- Cheesecake" >> menu.txt
git add menu.txt
git commit -m "Agregar producto al menu"
```

```
git switch main
git switch -c feature-estilos
echo "body { background: beige; }" > estilos.css
git add estilos.css
git commit -m "Agregar estilos iniciales"

git switch main
git merge feature-estilos
git merge feature-menu
git branch -d feature-estilos
git branch -d feature-menu

git log --oneline --graph --all
```

Resultado

Proyecto con dos ramas temporales, dos cambios integrados y un historial visible.

Checklist de aprendizaje

- Puedo explicar que problema resuelven las ramas.
- Puedo crear una rama con `git switch -c`.
- Puedo moverme entre ramas con `git switch`.
- Puedo fusionar una rama con `git merge`.
- Puedo resolver un conflicto basico.
- Puedo eliminar ramas con `git branch -d`.
- Puedo explicar por que Git se detiene cuando dos ramas cambian la misma linea.

Resumen de la sesion

- 1 Las ramas permiten trabajar en paralelo sin romper la linea principal.
- 2 Una rama es un puntero ligero que nace desde un commit existente.
- 3 `git merge` integra en la rama actual cambios que venian de otra rama.
- 4 Los conflictos no son errores: son decisiones que Git no puede tomar por nosotros.
- 5 Las ramas temporales deben limpiarse despues de integrarse.
- 6 Para trabajo compartido, prioriza claridad, estados limpios y mensajes de commit utiles.

Para practicar en casa

Ejercicio 1

En Cafe Aroma, crea `feature-menu` y `feature-estilos`. Haz un commit en cada una y fusionalas a `main`.

Ejercicio 2

Provoca un conflicto editando el mismo `<h1>` desde `main` y desde `feature-header`. Resuélvelo combinando ambas ideas.

Ejercicio 3

Elimina las ramas fusionadas con `git branch -d` y confirma que solo queda `main`.

Historial y deshacer cambios

Lo que no repetiremos hoy

Sesión 01

- Teoría, terminal e instalación.
- `git init`, `status`, `add`, `commit`.
- Staging, HEAD, `restore` básico y `.gitignore`.

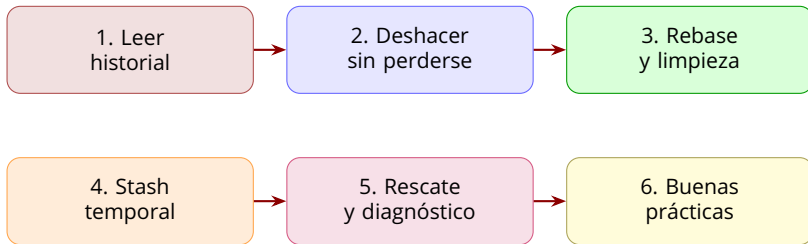
Sesión 02

- Ramas con `branch` y `switch`.
- `merge`, conflictos y limpieza de ramas.
- Laboratorio completo con ramas temporales.

Objetivo de hoy

Cerrar Git como herramienta local: leer historial, deshacer con criterio, reorganizar trabajo y usar comandos de rescate.

Mapa de la sesión final de Git



Leer el historial como un mapa

Comandos

```
git log
git log --oneline
git log --oneline --graph --all
git show <hash>
```

- log responde: ¿qué pasó?
- -graph -all muestra ramas y fusiones.
- show abre un commit concreto: mensaje, autor y cambios.

Regla práctica

Antes de deshacer o mover historial, mira primero el mapa.

Comparar cambios con diff

Preguntas frecuentes

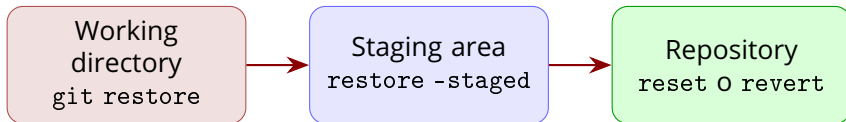
```
git diff
git diff --staged
git diff main..feature
git diff <hash1> <hash2>
```

- `git diff`: cambios no preparados.
- `git diff --staged`: cambios que entrarán al commit.
- Entre ramas o commits: compara dos puntos del historial.

Hábito profesional

Revisar diff antes de cada commit evita subir cambios accidentales.

Primero identifica dónde está el cambio



Idea central

No todos los cambios se deshacen igual: depende de si ya fueron preparados o confirmados.

Corregir antes del commit

Caso 1: archivo modificado

```
git restore archivo.txt
```

Descarta el cambio del directorio de trabajo.

Caso 2: archivo en staging

```
git restore --staged archivo.txt
```

Lo saca del próximo commit, sin borrar el contenido.

Antes de ejecutar

Usa `git status` y `git diff`. `restore` puede eliminar trabajo no confirmado.

Corregir el último commit

Cambiar mensaje

```
git commit --amend -m "Mensaje correcto"
```

Agregar algo olvidado

```
git add archivo.txt  
git commit --amend
```

- amend reemplaza el último commit por uno nuevo.
- Es ideal para corregir errores pequeños antes de compartir.
- No lo uses sobre historial que otras personas ya tienen.

reset: mover la rama hacia atrás

Comando	Conserva cambios	Uso típico
<code>git reset -soft HEAD~1</code>	En staging	Rehacer el commit
<code>git reset -mixed HEAD~1</code>	En working directory	Separar cambios
<code>git reset -hard HEAD~1</code>	No conserva	Descartar todo

Advertencia

-hard borra cambios locales. Ejecútalo solo si puedes explicar exactamente qué vas a perder.

revert: deshacer sin borrar historia

Comando

```
git revert <hash>
```

- Crea un nuevo commit que revierte otro commit.
- Es más seguro cuando el historial ya fue compartido.
- Deja visible la decisión: qué se hizo y qué se deshizo.

Comparación rápida

reset reescribe hacia atrás; revert avanza con una corrección.

Rebase y limpieza de historial

rebase: **cambiar la base de una rama**

- rebase toma los commits de una rama y los vuelve a aplicar encima de otra base.
- Sirve para actualizar una rama de trabajo cuando `main` ya avanzo.
- El resultado suele ser un historial mas lineal y facil de leer.

Idea clave

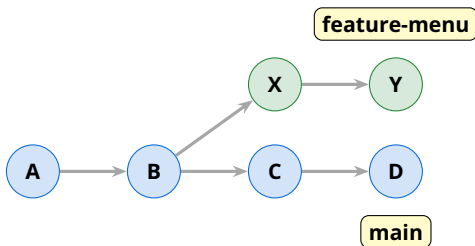
No mezcla historias como `merge`; reescribe la ubicacion de tus commits.



Reubicar commits

Mis cambios quedan encima del nuevo
`main`.

Escenario: mi rama quedo atras



Lectura

La rama feature-menu nacio en B, pero main ya avanzo hasta D. Antes de integrar, queremos probar nuestros cambios sobre la version mas reciente.

Comando basico de rebase

Actualizar mi rama

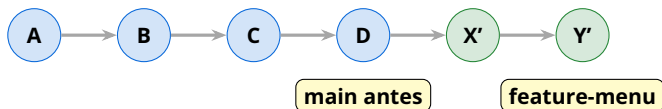
```
git switch feature-menu  
git rebase main
```

Despues de revisar

```
git switch main  
git merge feature-menu
```

- Primero te paras en tu rama de trabajo.
- Luego pides que se reubique encima de main.
- Si todo sale bien, el merge final suele ser directo.

Despues del rebase: historial lineal



Detalle importante

Los commits X' y Y' no son exactamente los mismos SHA que X y Y. Git crea nuevos commits equivalentes sobre otra base.

Conflictos durante rebase

Flujo de resolución

```
git rebase main
# resolver archivos con conflicto
git add archivo.txt
git rebase --continue
```

Cancelar la operación

```
git rebase --abort
```

- El conflicto se resuelve igual que en un merge: editar, probar y marcar como resuelto.
- `-continue` le dice a Git que siga aplicando commits.
- `-abort` vuelve al estado anterior al rebase.

merge **vs** rebase: dos formas de integrar

merge

- Conserva la historia real de integracion.
- Puede crear un commit de merge.
- Es claro para trabajo compartido.

rebase

- Reescribe commits encima de otra base.
- Deja una linea de historial mas limpia.
- Es util para ordenar tu rama antes de integrarla.

Regla de oro

No hagas rebase sobre commits que otras personas ya tienen. Reescribir historial compartido genera confusion y conflictos innecesarios

Cuando usar cada uno

Situacion	Mejor opcion	Motivo
Integrar una funcionalidad terminada a <code>main</code>	<code>merge</code>	Conserva el punto de integracion.
Actualizar mi rama local con cambios recientes de <code>main</code>	<code>rebase</code>	Prueba mi trabajo sobre la base nueva.
Historial ya compartido con el equipo	<code>merge</code> o <code>revert</code>	Evita reescribir commits ajenos.
Limpiar commits propios antes de compartir	<code>rebase</code>	Deja una historia mas legible.
Deshacer algo que ya llego al equipo	<code>revert</code>	Corrige sin borrar la historia.

Buenas practicas de historial

- Ejecuta `git status` antes de operaciones importantes.
- Revisa `git diff` antes de cada commit.
- Haz commits atomicos: un cambio logico por commit.
- Escribe mensajes claros en modo imperativo.

Buenos mensajes

Agrega validacion de email
Corrige enlace del menu
Actualiza README inicial

Buenas practicas de ramas y seguridad

Ramas

- Usa nombres claros: `feature/menu`, `fix/navbar`, `docs/readme`.
- Mantén ramas temporales pequeñas.
- Elimina ramas fusionadas con `git branch -d`.

Seguridad

- No subas `.env`, claves, tokens ni contraseñas.
- Usa `.gitignore` para archivos locales.
- Prefiere `revert` si el historial ya fue compartido.

Repaso: que comando elegir

Necesidad	Comando	Cuidado principal
Ver estado del repo	<code>git status</code>	Hazlo siempre primero.
Ver cambios no preparados	<code>git diff</code>	Revisa antes de agregar.
Sacar del staging	<code>restore -staged</code>	No borra contenido.
Corregir ultimo commit propio	<code>commit -amend</code>	No si ya fue compartido.
Retroceder commits locales	<code>reset</code>	-hard borra trabajo.
Deshacer historial compartido	<code>revert</code>	Crea un commit nuevo.
Actualizar rama propia	<code>rebase main</code>	No sobre commits ajenos.
Integrar trabajo terminado	<code>merge</code>	Resolver conflictos si aparecen.

Laboratorio integrador: Cafe Aroma final

```
mkdir cafe-aroma-final
cd cafe-aroma-final
git init -b main
echo "<h1>Cafe Aroma</h1>" > index.html
echo "- Cafe americano" > menu.txt
git add .
git commit -m "Proyecto inicial de cafeteria"
```

```
git switch -c feature-menu
echo "- Cheesecake" >> menu.txt
git add menu.txt
git commit -m "Agrega cheesecake al menu"
echo "- Capuccino" >> menu.txt
git add menu.txt
git commit -m "Agrega capuccino al menu"
```

```
git switch main
echo "<p>Abierto de lunes a viernes</p>" >> index.html
git add index.html
git commit -m "Agrega horario de atencion"
```

```
git switch feature-menu
git log --oneline --graph --all
git rebase main
```

```
git switch main
git merge feature-menu
git branch -d feature-menu
git log --oneline --graph --all
```

Reto practico individual

- 1 Crea una rama `feature-contacto` desde `main`.
- 2 Agrega una seccion de contacto en `index.html` y haz un commit atomico.
- 3 En `main`, modifica otra parte del mismo archivo y confirma el cambio.
- 4 Actualiza tu rama con `git rebase main`.
- 5 Si aparece conflicto, resuelvelo y usa `git rebase -continue`.
- 6 Integra la rama a `main` y elimina la rama temporal.
- 7 Muestra el resultado con `git log -oneline -graph -all`.

Checklist final de Git

- Puedo explicar el flujo: modificar, preparar y confirmar.
- Puedo leer historial con `log`, `show` y `diff`.
- Puedo trabajar con ramas, `merge` y conflictos.
- Puedo deshacer con `restore`, `amend`, `reset` y `revert`.
- Puedo explicar la diferencia entre `merge` y `rebase`.
- Puedo elegir una estrategia segura segun si el historial es local o compartido.

Resumen de la sesion

- 1 Git no es memorizar comandos: es entender estados e historial.
- 2 `merge` integra historias; `rebase` reorganiza commits propios.
- 3 `reset` reescribe hacia atras; `revert` corrige avanzando.
- 4 Antes de deshacer o mover historial: `git status`, `git diff` y `git log`.
- 5 Buenas practicas: commits atomicos, ramas claras, historial legible y nada de secretos.
- 6 Con esta base ya podemos pasar a GitHub: remotos, `push`, `pull` y Pull Requests.